- Write clean, modular, and well-documented code.
- Focus on meaningful and interpretable visualizations.
- Ensure the program handles edge cases, such as missing data or invalid file inputs.

Instructions:

1. Write clean, modular, and well-commented code.
2. Display meaningful outputs and prompts for user interaction.
3. Demonstrate OOP concepts and ensure all functionalities are implemented.
4. Submit your public GitHub repository containing a Python file (.py) and other necessary files (such as datasets if any)

missing or invalid entries.

2. **OOP Design**
   - Encapsulate functionalities into a `LibraryDashboard` class with reusable and modular methods.

3. **NumPy Integration**
   - Use NumPy arrays for calculations like average borrowing duration and the standard deviation of borrow times.
4. **Pandas for Data Handling**
   - Use Pandas for data cleaning, aggregation, and manipulation tasks.
   - Create new computed columns, such as late returns or frequent borrowers.
5. **Data Visualization**
   - Use Matplotlib and Seaborn to create professional and informative graphs.
   - Include appropriate labels, titles, and legends for all visualizations.

Submission Requirements

- Submit a Python script named `library_dashboard.py`.
- Attach the `library_transactions.csv` dataset used for testing.
- Include a `README.md` file with setup instructions and a description of the project.

Example Workflow

1. **Input Validation**:
   - Prompt the user to upload the `library_transactions.csv` file.
   - Validate the file format and handle missing or incorrect data.
2. **Data Loading**:
   - Load the dataset into a Pandas DataFrame using the `load_data()` method.
3. **Statistics Calculation**:
   - Identify the most borrowed books and the average borrowing duration using `calculate_statistics()`.
4. **Data Filtering**:
   - Allow users to filter data by genre, date range, or borrowing duration using `filter_transactions()`.
5. **Visualizations**:
   Generate bar charts, line graphs, pie charts, and heatmaps for analysis using Matplotlib and Seaborn.

Key Notes for Students

ur productivity with AI, plus get 4x usage limits in the Gemini app, and 5 TB of storage for 74% off for 3 months

Request edit access

SINCE 2008
RED & WHITE
SKILL EDUCATION
SHAPING SKILLS FOR SCALING HIGHER...!!!

Project Title: "E-Library Data Insights Dashboard"

Objective

Develop a Python-based **E-Library Data Insights Dashboard** to manage and analyze book borrowing trends. The project should integrate **Control Structures, Arrays, OOP, NumPy, Pandas with Datasets, and Matplotlib & Seaborn** for data manipulation and visualization.

Problem Statement

You are tasked with creating an **E-Library Data Insights Dashboard** to process and analyze library usage data. The dashboard must provide insights into book popularity, user activity, and borrowing trends over time.

Features and Functionalities

1. Data Input and Validation (Control Structure & Arrays - Low Weightage)

- Accept a CSV file as input containing library transaction data.
- Validate the file format and check for missing or inconsistent data using control

- Handling missing or duplicate data entries.

4. Data Visualization (Matplotlib & Seaborn - High Weightage)

- Create insightful visualizations to analyze library trends:
  - **Bar Chart**: Top 5 most borrowed books.
  - **Line Graph**: Borrowing trends over months.
  - **Pie Chart**: Distribution of books borrowed by genre.
  - **Heatmap**: Borrowing activity by day and time.

Dataset Details

Use(by generating with the help of AI) a sample CSV file named `library_transactions.csv`, containing the following columns:

- `Transaction ID`
- `Date` (YYYY-MM-DD)
- `User ID`
- `Book Title`
- `Genre`
- `Borrowing Duration (Days)`

Implementation Guidelines

1. **Control Structures & Arrays**

and more Boost your productivity with AI, plus get 4x usage limits in the Gemini app

Request edit access

Help

structures.

2. Library Management System (OOP - Medium Weightage)

- Design a `LibraryDashboard` class with the following methods:
  - `load_data(self, file_path)`: Load and validate the dataset using Pandas.
  - `calculate_statistics(self)`: Calculate metrics like the most borrowed book, average borrowing time, and the busiest day.
  - `filter_transactions(self, condition)`: Filter transactions based on user-defined criteria like date range or genre.
  - `generate_report(self)`: Generate a summary report of the analysis.

3. Data Analysis and Computations (NumPy & Pandas - Medium Weightage)

- Use **NumPy** to calculate borrowing duration statistics.
- Use **Pandas** for:
  - Grouping data by book genre or user type.
  - Aggregating metrics such as total borrowings per book or average borrowing duration.